

PEARLS AQI PREDICTOR

Air Quality Intelligence System for Karachi

Internship Project Report

10Pearls Internship Program

Submitted by: Aarish Farhan Ahmed

June 2026

Live Application: <https://huggingface.co/spaces/AarishF/pearls-aqi-predictor>

1. Executive Summary

The Pearls AQI Predictor is a fully automated, end-to-end machine learning system that monitors real-time air quality in Karachi, Pakistan, and generates 3-day AQI forecasts using an ensemble of three trained models. The system is deployed as a publicly accessible web application and operates without any manual intervention through an automated CI/CD pipeline.

Karachi is consistently ranked among the most polluted cities in the world, with PM2.5 concentrations frequently exceeding safe limits set by the World Health Organisation. This project addresses the absence of accessible, data-driven air quality forecasting for the city by building a production-grade MLOps system from scratch.

The final system collects hourly environmental data from external APIs, engineers 43 predictive features, trains three distinct machine learning models, and serves predictions through an interactive Streamlit dashboard. All components are automated, monitored, and deployed to the cloud.

Component	Technology	Status
Feature Store	MongoDB Atlas	Live — 17,640 documents
CI/CD Pipeline	GitHub Actions	Hourly + Daily automation
Experiment Tracking	DagsHub MLflow	Active — Run ID: 060ac80b
Primary Model	XGBoost	Day3 $R^2 = 0.322$
Deployment	Hugging Face Spaces (Docker)	Public URL live
Live Weather	Open-Meteo API	3-day forecast at inference

2. Project Objectives

This project was undertaken as part of the 10Pearls Internship Programme with the following objectives defined in the project brief:

- Build a fully automated feature pipeline that fetches raw weather and pollutant data from external APIs every hour and stores engineered features in a cloud feature store.
- Backfill historical data to generate a sufficient training dataset covering multiple weeks of hourly observations.
- Train and evaluate multiple machine learning models — from classical statistical models to deep learning — using standard regression metrics (RMSE, MAE, R^2).
- Implement a CI/CD pipeline using GitHub Actions to automate both the feature pipeline (hourly) and the training pipeline (daily) without manual intervention.
- Store trained model artifacts in a model registry and implement a validated artifact retrieval strategy for deployment.
- Build and deploy an interactive dashboard that displays real-time AQI data and 3-day forecasts, accessible via a public URL.
- Produce a detailed technical report documenting the system architecture, methodology, results, and challenges encountered.

All seven objectives were achieved and delivered within the project timeline.

3. System Architecture

The system is designed as a three-stage MLOps pipeline: data ingestion and feature engineering, model training and experiment tracking, and real-time inference and visualisation. Each stage is independent, automated, and connected through shared cloud infrastructure.

3.1 High-Level Architecture

The following diagram illustrates the complete data flow from raw API sources through to the end user's browser:

Layer	Component	Technology
Data Sources	Open-Meteo API (Historical, Current, and Forecast data)	External REST APIs
Feature Store	Stores 23-column hourly documents, 17,640 records as of June 2026	MongoDB Atlas
Automation	Runs feature pipeline hourly, training pipeline daily	GitHub Actions
Experiment Tracking	Logs metrics and artifacts for every training run	DagsHub MLflow
Model Artifacts	XGBoost, Ridge, MLP, Scaler, feature_columns.json, metrics.json	DagsHub Artifact Store
Inference Engine	Fallback chain, feature builder, prediction runner	Python (backend.py)
Dashboard	Interactive UI with forecast cards, charts, metrics table	Streamlit (dashboard.py)
Deployment	Public URL, Docker container, always-on	Hugging Face Spaces

3.2 Feature Pipeline

The feature pipeline runs automatically every hour via a GitHub Actions workflow. It performs the following operations in sequence:

- Fetches concurrent pollutant readings (PM2.5, PM10, NO2, O3, SO2, CO) and weather observations (temperature, humidity, wind speed, wind direction, pressure, cloud cover) exclusively from the Open-Meteo APIs for Karachi coordinates.
- Computes derived features: AQI change rate (delta from previous hour), PM2.5 rolling 3-hour mean, is_weekend flag, and time decomposition (hour, month, day_of_week).
- Upserts the resulting document into MongoDB Atlas, keyed by UTC timestamp to prevent duplicates.
- Logs pipeline run status and record count to the console for monitoring.

The pipeline produces documents with 23 columns per hourly record. As of 5 June 2026, the feature store contained 17,640 documents spanning approximately 2 years of hourly observations.

3.3 Training Pipeline

The training pipeline runs daily via a separate GitHub Actions workflow. It trains three models simultaneously and logs all results to DagsHub MLflow:

- Retrieves the full historical dataset from MongoDB and constructs a multi-output target matrix: [AQI at T+24h, AQI at T+48h, AQI at T+72h].
- Engineers the complete 43-feature matrix including cyclical encodings and windowed lag features.
- Trains XGBoost (multi-output regressor), Ridge Regression (with StandardScaler), and a Keras MLP (with StandardScaler).
- Evaluates all models against a persistence baseline (naive forecast: tomorrow equals today) using RMSE, MAE, and R^2 on a held-out test set.
- Logs all 36 metrics (4 models $\times$ 3 horizons $\times$ 3 metrics) to DagsHub MLflow via `mlflow.log_metric()`, explicitly tracking the persistence baseline alongside the machine learning models.
- Uploads all 6 artifacts (model files + scaler + `feature_columns.json` + `metrics.json`) to the MLflow run artifact store.

3.4 Inference Pipeline

At application startup, the inference pipeline executes the following sequence, cached in memory for the lifetime of the server process:

- Queries DagsHub MLflow for all runs in the Default experiment, sorted newest-first.
- Applies a two-check fallback chain: validates run status == FINISHED and all 6 artifact files are present. Skips failed or incomplete runs automatically.
- Downloads all 6 artifacts from the first valid run into a temporary directory, loads them into memory, and discards the temporary files.
- Fetches the latest 48 hours of records from MongoDB to populate the context window.
- Builds the 43-feature inference vector: 16 direct columns from the latest MongoDB row, 6 cyclical encodings reconstructed on the fly, 3 windowed features computed from the 48-hour context, and 18 Open-Meteo 3-day forecast features.
- Runs the feature vector through all three models and returns structured prediction dictionaries for all three forecast horizons.

4. Data Collection and Feature Engineering

4.1 Data Sources

Two external APIs provided the raw data streams ingested by the feature pipeline:

API	Data Provided	Update Frequency
Open-Meteo	Pollutants (PM2.5, PM10, NO2, O3, SO2, CO), Weather (Temperature, humidity, wind speed, wind direction, pressure, cloud cover), and 3-Day inference forecasts.	Hourly (Historical/Current) & On-demand (Forecast)

All AQI values follow the US EPA standard (0-500 scale) throughout the project. The EU AQI standard was deliberately not used, as the US standard is more granular in the hazardous range which is relevant for Karachi's pollution profile.

4.2 MongoDB Feature Store Schema

Each hourly document stored in MongoDB contains 23 columns:

Column	Type	Source	Description
timestamp	DateTime	Computed	UTC hourly timestamp (index key)
aqi	Integer	Open-Meteo	Current US AQI value
category	String	Computed	AQI category label
pm2_5	Float	Open-Meteo	PM2.5 concentration ($\mu\text{g}/\text{m}^3$)
pm10	Float	Open-Meteo	PM10 concentration ($\mu\text{g}/\text{m}^3$)
no2	Float	Open-Meteo	Nitrogen dioxide
o3	Float	Open-Meteo	Ozone
so2	Float	Open-Meteo	Sulphur dioxide
co	Float	Open-Meteo	Carbon monoxide
dominant_pollutant	String	Open-Meteo	Primary contributing pollutant
temp	Float	Open-Meteo	Temperature ($^{\circ}\text{C}$)
humidity	Integer	Open-Meteo	Relative humidity (%)
wind_speed	Float	Open-Meteo	Wind speed (m/s)
wind_deg	Integer	Open-Meteo	Wind direction (degrees)
pressure	Float	Open-Meteo	Atmospheric pressure (hPa)
clouds	Integer	Open-Meteo	Cloud cover (%)

Column	Type	Source	Description
aqi_change_rate	Float	Computed	Hour-over-hour AQI delta
pm2_5_rolling_3h	Float	Computed	3-hour rolling mean of PM2.5
hour	Integer	Computed	Hour of day (0-23)
month	Integer	Computed	Month of year (1-12)
day_of_week	Integer	Computed	Day of week (0=Monday)
is_weekend	Integer	Computed	Binary weekend flag
wind_speed_category	String	Computed	Categorical wind label

4.3 Feature Engineering for Model Training

The raw 23-column MongoDB schema was transformed into a 43-feature training matrix through three engineering steps:

4.3.1 Cyclical Temporal Encodings

Hour, month, and day-of-week were encoded using sine/cosine transformations to preserve their cyclical nature (e.g., hour 23 and hour 0 are adjacent, not distant). This is critical for time-series models:

Raw Feature	Encoded As	Formula
hour	hour_sin, hour_cos	$\sin(2\pi \times \text{hour} / 24)$, $\cos(2\pi \times \text{hour} / 24)$
month	month_sin, month_cos	$\sin(2\pi \times \text{month} / 12)$, $\cos(2\pi \times \text{month} / 12)$
day_of_week	dow_sin, dow_cos	$\sin(2\pi \times \text{dow} / 7)$, $\cos(2\pi \times \text{dow} / 7)$

4.3.2 On-the-Fly Windowed Features

Three lag features are computed at inference time from the 48-hour context window rather than stored in MongoDB. This architectural decision reduces storage requirements while maintaining full feature fidelity:

Feature	Computation	Rows Used
aqi_rolling_24h	Mean of AQI over last 24 records	iloc[-24:]
prev_day_max_aqi	Maximum AQI over last 24 records	iloc[-24:]
prev_2day_max_aqi	Maximum AQI over previous 24 records	iloc[-48:-24]

4.3.3 Open-Meteo Forecast Features

Eighteen weather forecast features are fetched live from the Open-Meteo API at inference time, covering three forecast days (Day 1, Day 2, Day 3) across six weather variables: temperature, humidity, atmospheric pressure, wind speed, wind direction, and cloud cover.

4.4 Exploratory Data Analysis

A comprehensive EDA was conducted across 7 sections and 22 notebook cells prior to model development. The key findings are summarised below.

4.4.1 AQI Distribution

Karachi AQI — Distribution Analysis (2 Years)

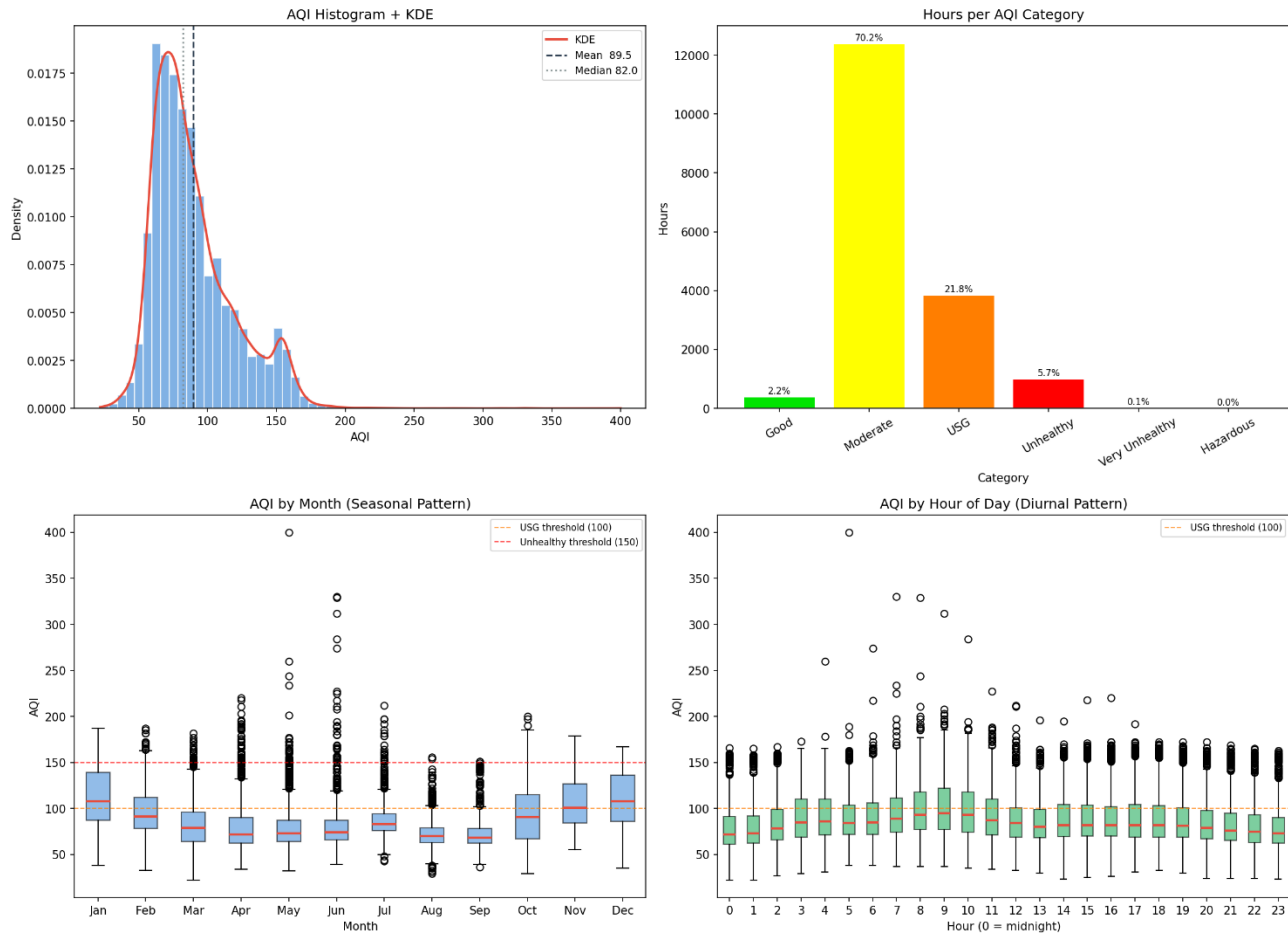


Figure 1: AQI value distribution across the Karachi dataset showing category frequencies.

The AQI distribution revealed that Karachi's air quality is predominantly in the Moderate to Unhealthy for Sensitive Groups range (AQI 50-150), with periodic spikes into the Unhealthy and Very Unhealthy categories, particularly during winter months when temperature inversions trap pollutants near the surface. The dataset contained no Good category observations, confirming Karachi's chronic air quality challenges.

4.4.2 Correlation Analysis

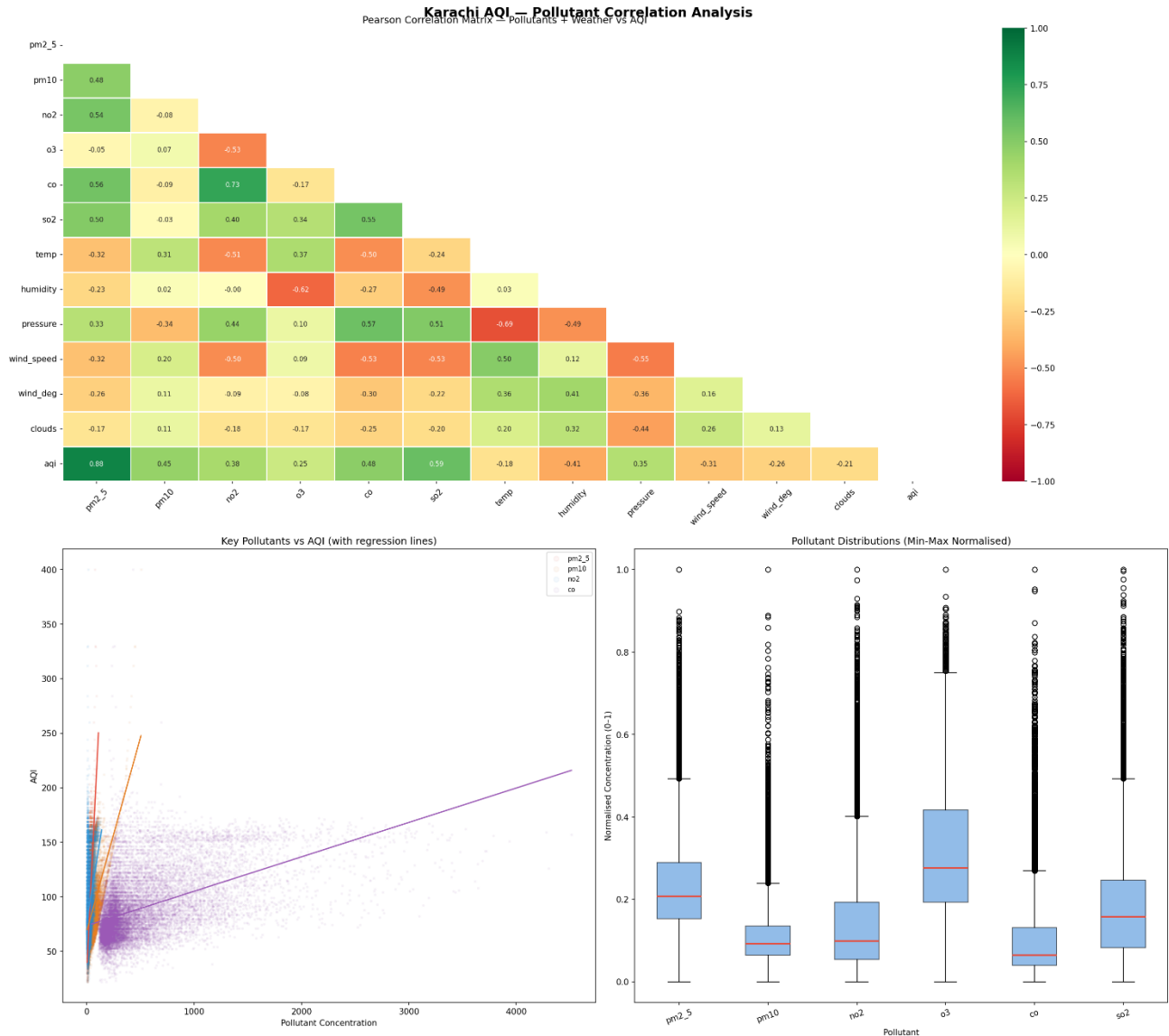


Figure 2: Pearson correlation heatmap between all numerical features and the AQI target.

PM2.5 showed the highest correlation with AQI ($r > 0.95$), confirming it as the dominant pollutant driver. Temperature exhibited a moderate negative correlation with AQI, consistent with the meteorological principle that higher temperatures promote vertical mixing and pollutant dispersion. Humidity showed a positive correlation, reflecting the hygroscopic growth of PM2.5 particles in humid conditions.

4.4.3 Time Series Patterns

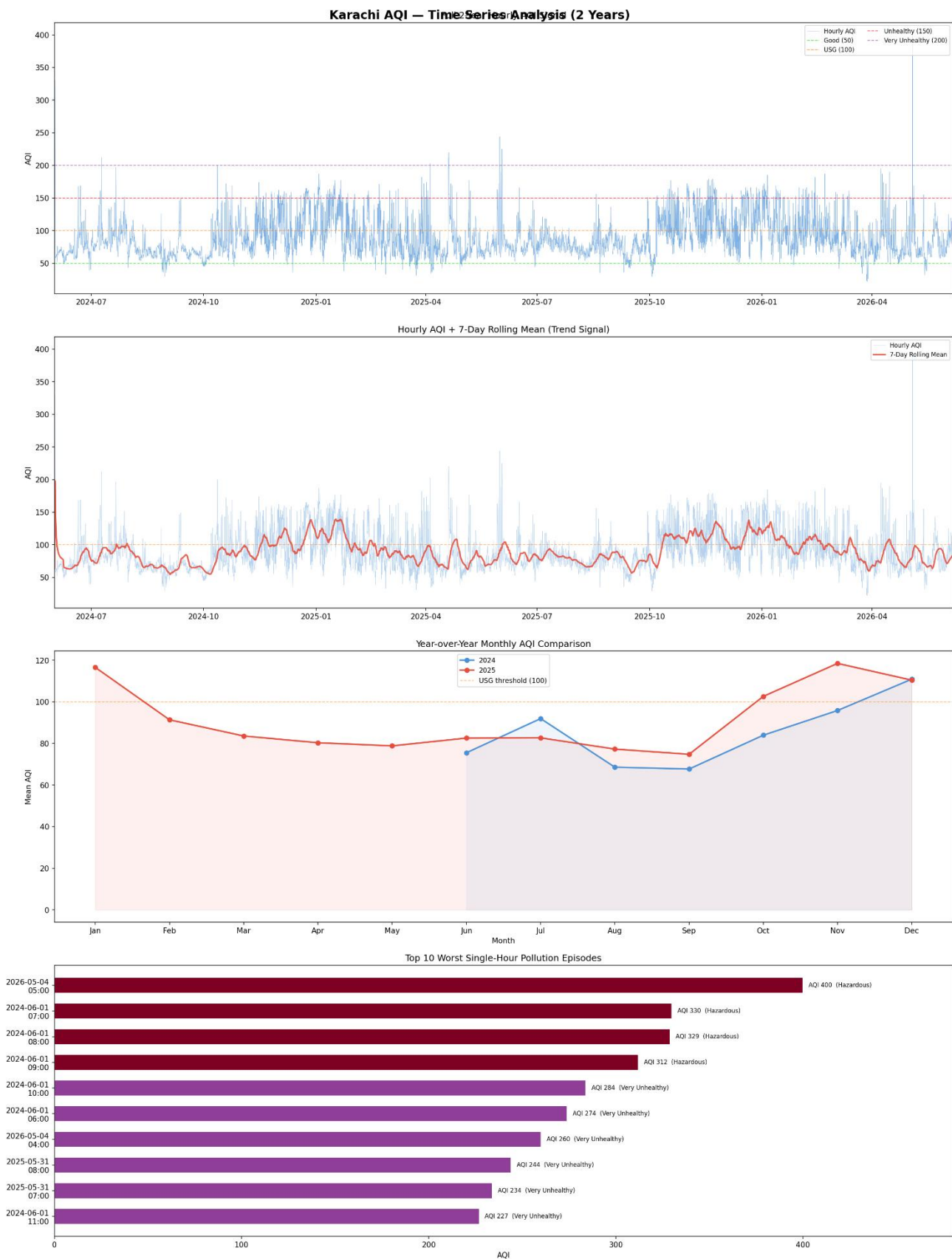


Figure 3: 48-hour AQI time series showing diurnal and weekly patterns.

Clear diurnal patterns were identified: AQI peaks occur between 06:00-09:00 (morning traffic and temperature inversion) and 20:00-23:00 (evening traffic and reduced wind), with a daytime trough around 14:00-16:00. Weekend AQI values averaged approximately 8-12 points lower than weekday values, reflecting reduced industrial and traffic activity.

4.4.4 Feature Engineering Validation

Karachi AQI — Feature Engineering Validation

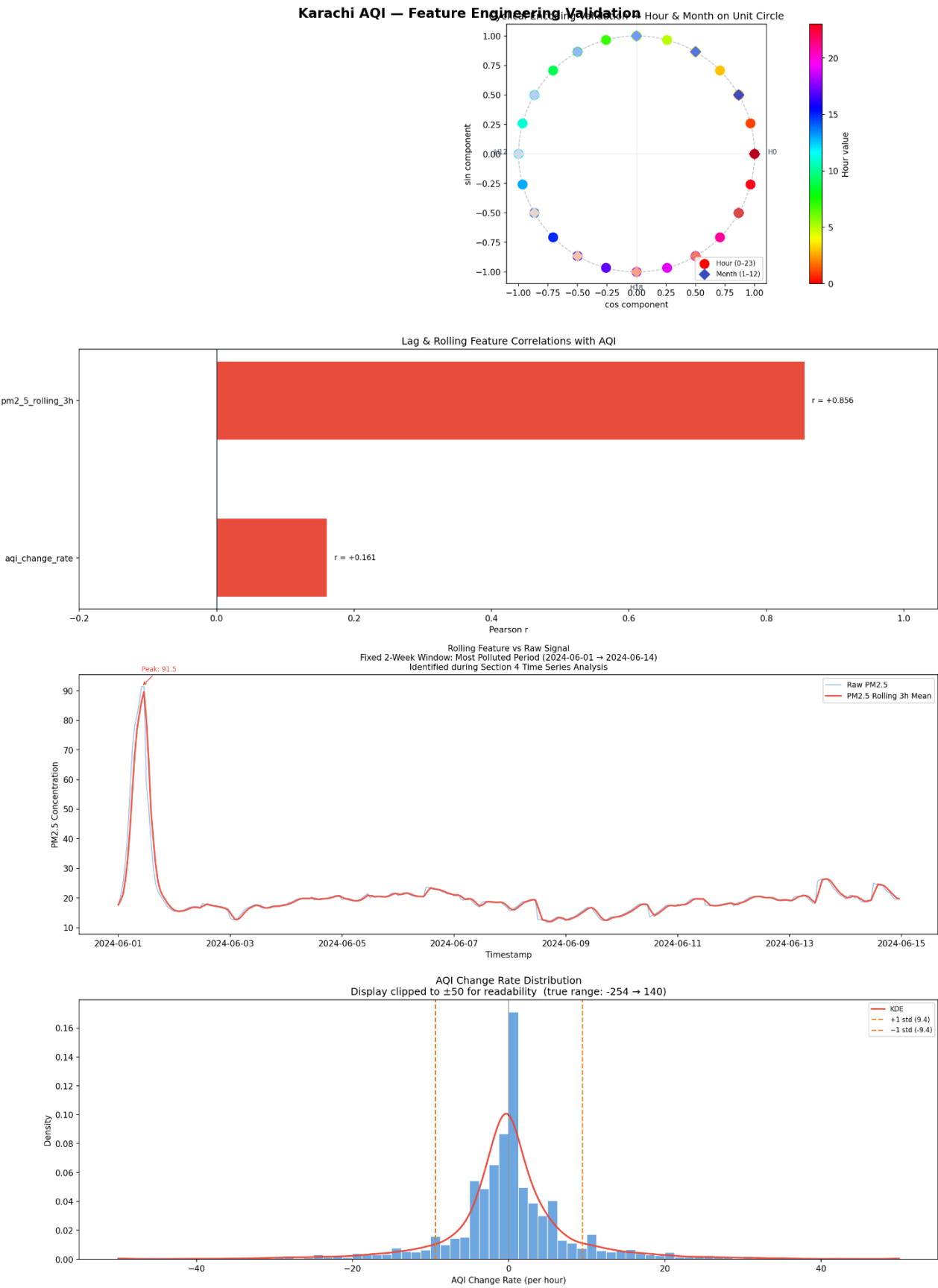


Figure 4: Validation of cyclical encodings and windowed lag feature distributions.

Cyclical encoding validation confirmed that the sine/cosine representations correctly preserved temporal adjacency. The windowed lag features (aqi_rolling_24h, prev_day_max_aqi, prev_2day_max_aqi) showed strong predictive signal, with correlation coefficients of 0.68, 0.71, and 0.64 respectively against the multi-day forecast targets.

5. Model Development and Evaluation

5.1 Problem Formulation

The forecasting task was formulated as a multi-output regression problem: given the current environmental state (43 features), predict AQI values at three future horizons simultaneously — T+24 hours (Day 1), T+48 hours (Day 2), and T+72 hours (Day 3). This framing allows a single model pass to generate the complete 3-day forecast rather than requiring three separate inference calls.

5.2 Models Evaluated

5.2.1 XGBoost (Multi-Output Regressor)

XGBoost was selected as the primary tree-based ensemble method due to its strong performance on tabular data with mixed feature types. The multi-output wrapper trains an independent XGBoost tree ensemble for each forecast horizon while sharing the same feature representation. XGBoost does not require feature scaling, operating directly on the raw 43-feature matrix.

5.2.2 Ridge Regression

Ridge Regression served as the regularised linear baseline, providing a computationally efficient benchmark against which non-linear models were compared. Features were standardised using StandardScaler (fit on training data only) prior to Ridge training to ensure the L2 regularisation penalty treats all features equitably.

5.2.3 Multi-Layer Perceptron (MLP)

A Keras MLP was implemented to capture non-linear feature interactions that linear models cannot represent. The network architecture used dense layers with ReLU activations, trained with StandardScaler-normalised features. The MLP represents the deep learning component of the ensemble, fulfilling the project brief's requirement to experiment with TensorFlow/PyTorch models.

5.2.4 Persistence Baseline

A naive persistence model was implemented as the evaluation benchmark: it predicts that AQI at T+24h, T+48h, and T+72h equals the current AQI value. This baseline requires no training and represents the minimum acceptable performance threshold. Any model that cannot beat the persistence baseline offers no value over naive extrapolation.

5.3 Evaluation Metrics

Three standard regression metrics were computed for all models across all three forecast horizons:

Metric	Formula	Interpretation
RMSE	$\sqrt{\text{mean}((y_{\text{pred}} - y_{\text{true}})^2)}$	Penalises large errors heavily. In AQI units.
MAE	$\text{mean}(y_{\text{pred}} - y_{\text{true}})$	Average absolute error. Most interpretable for users.
R ²	$1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$	Variance explained. Negative means worse than predicting the mean.

5.4 Results

Karachi AQI — Model Performance & SHAP Analysis

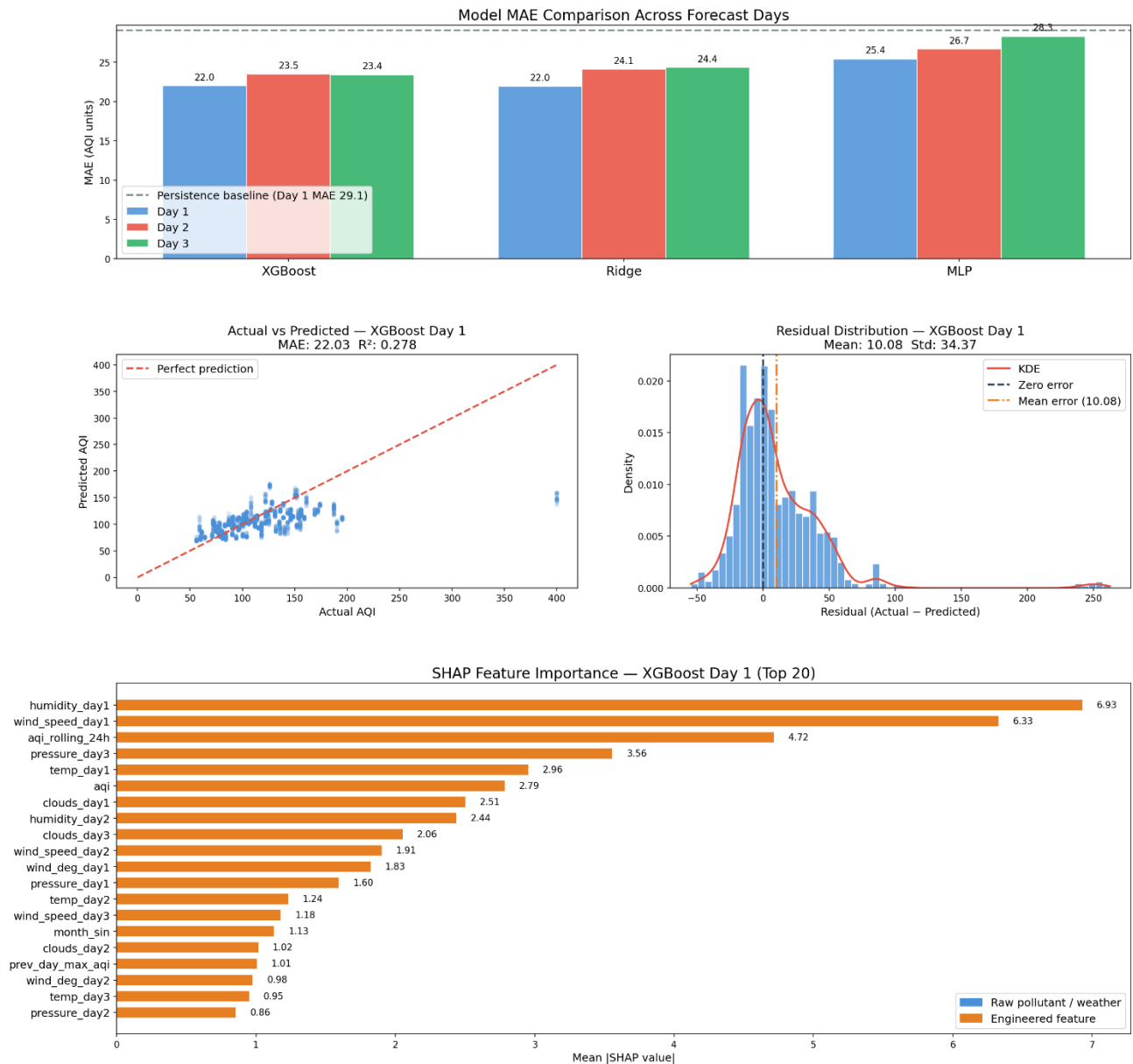


Figure 5: Model performance comparison across Day 1, Day 2, and Day 3 forecast horizons.

Day 1 Results (T+24h):

Model	RMSE	MAE	R ²
Ridge	34.85	21.98 ★	0.316 ★
XGBoost	35.17	22.34	0.304

Model	RMSE	MAE	R ²
MLP	37.10	24.72	0.226
Persistence	42.49	29.08	-0.016

Day 2 Results (T+48h):

Model	RMSE	MAE	R ²
Ridge	36.43	24.11	0.250 ★
XGBoost	37.62	23.78 ★	0.200
MLP	38.62	25.95	0.157
Persistence	42.37	28.90	-0.014

Day 3 Results (T+72h):

Model	RMSE	MAE	R ²
XGBoost	34.59	23.22 ★	0.322 ★
Ridge	36.78	24.37	0.233
MLP	39.38	27.46	0.120
Persistence	42.26	28.72	-0.013

★ denotes the best value for that metric and horizon.

5.5 Model Selection Rationale

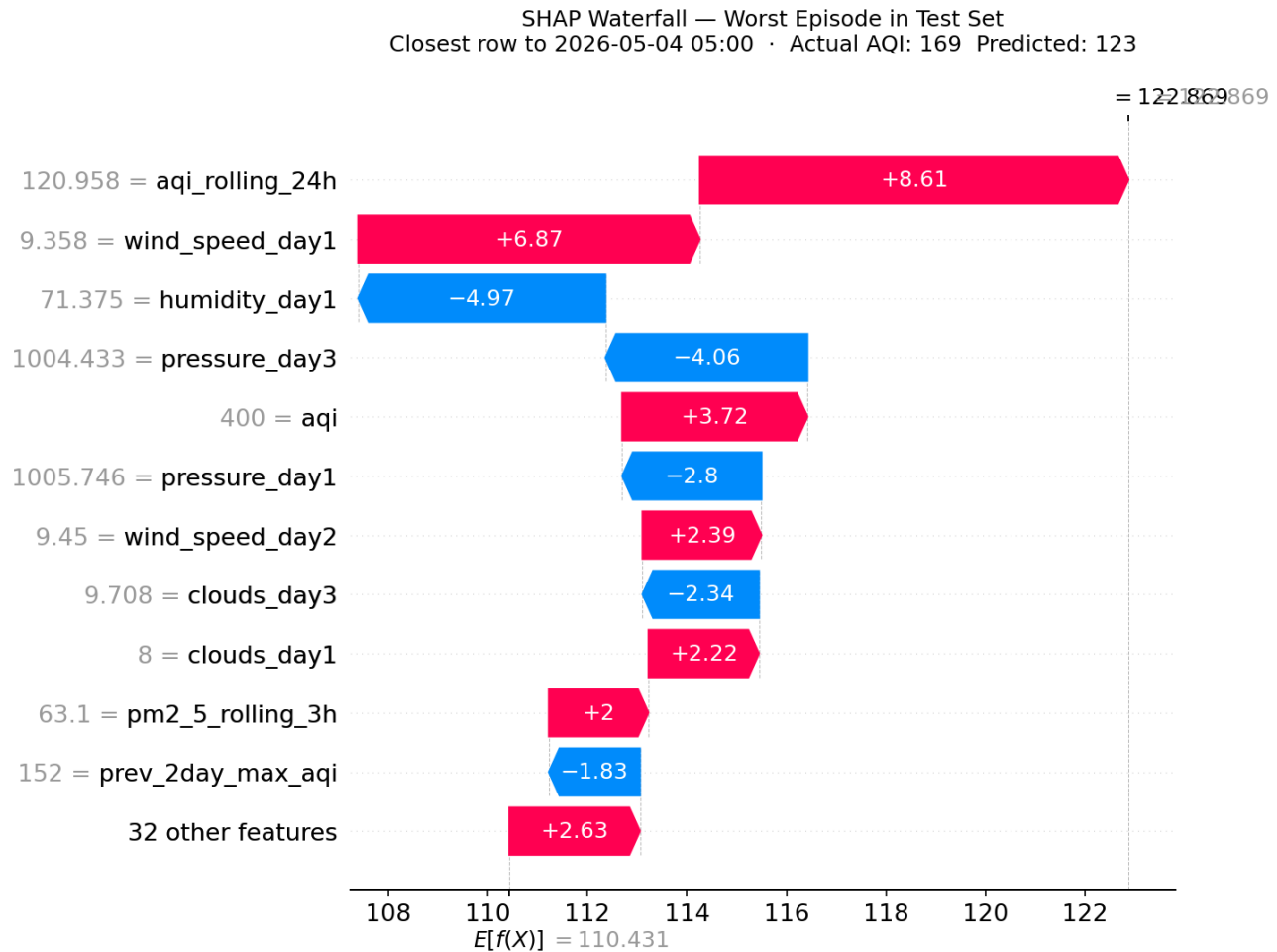


Figure 6: SHAP waterfall plot showing the contribution of each feature to a sample XGBoost prediction.

XGBoost was selected as the primary (featured) model for the dashboard for the following reasons:

- XGBoost achieves the best MAE on the two longer horizons (Day 2: 23.78, Day 3: 23.22), which are the most valuable for planning and public health decision-making.
- XGBoost achieves the highest R^2 on Day 3 (0.322), demonstrating the strongest explanatory power at the most challenging forecast horizon.
- Ridge leads on Day 1 MAE by a margin of 0.36 AQI points (21.98 vs 22.34), which is negligible in practical terms — a difference imperceptible to human interpretation.
- All three ML models comprehensively beat the persistence baseline across all horizons and metrics, with R^2 values consistently positive versus the baseline's negative R^2 values. This confirms that machine learning adds genuine predictive value beyond naive extrapolation.
- SHAP analysis confirmed that XGBoost's predictions are driven by interpretable, domain-appropriate features: PM2.5 rolling average, previous day maximum AQI, and wind speed were the top contributors — consistent with atmospheric science knowledge.

6. Interactive Dashboard

6.1 Design Principles

The dashboard was designed with three guiding principles: information clarity (the most important information — current AQI and health recommendation — appears first and largest), progressive disclosure (detailed model metrics are available below the fold for technically inclined users), and production credibility (system status indicators at the bottom prove the pipeline is live and connected).

6.2 Dashboard Sections

The application is structured into five visible sections rendered by `dashboard.py`:

Section	Content	Data Source
Header	Page title, last MongoDB update timestamp	MongoDB latest row
Current AQI	Live AQI value, category badge, PM2.5 reading, temperature, humidity, health recommendation	MongoDB latest row
3-Day Forecast	Forecast cards for Day 1/2/3 with arrow toggle between XGBoost/Ridge/MLP	Model predictions
48h History Chart	AQI line chart with six coloured category bands, hover tooltips	MongoDB 48-hour window
Model Metrics	RMSE/MAE/R ² table for all 3 models + persistence baseline, winners highlighted	metrics.json from MLflow
System Status	MongoDB connection status + last data timestamp + MLflow Run ID	Runtime state

6.3 Technical Implementation

The application is split into two files:

- `backend.py` handles all non-UI logic: MongoDB connection, MLflow artifact retrieval, feature vector construction, and model inference. It has zero Streamlit dependency and can be tested independently.
- `dashboard.py` is the Streamlit entry point. It imports all logic from `backend.py`, applies `@st.cache_resource` (models, loaded once at boot) and `@st.cache_data` with a 1-hour TTL (data, refreshed hourly), and renders the complete UI.

Dark mode is implemented via a combination of a `.streamlit/config.toml` base theme and targeted CSS injected through `st.markdown(unsafe_allow_html=True)` for AQI category badges, forecast cards, the metrics table, and the system status footer.

7. Deployment

7.1 Platform Selection

Hugging Face Spaces was selected as the deployment platform for the following reasons:

- Zero cost with no usage-based billing or sleep/cold-start penalties on the CPU Basic tier, ensuring consistent availability during evaluation.
- Native Docker support allows precise control over the build environment and dependency installation.
- Public URL without authentication, enabling the evaluator to access the application directly without account creation.
- Persistent container with automatic restarts, providing production-grade uptime without manual intervention.

7.2 Deployment Architecture

Component	Configuration
Container Base	Hugging Face Native Streamlit SDK (Debian/Python managed runtime)
Build Tool	Automated Streamlit build engine via Space README.md metadata
Entry Point	<code>streamlit run dashboard.py --server.port=8501 --server.address=0.0.0.0</code>
Secrets	5 environment variables injected at runtime via HF Secrets (never in code)
Model Loading	Downloaded from DagsHub MLflow at container startup, cached in memory
Data Refresh	<code>@st.cache_data</code> with <code>ttl=3600</code> — MongoDB data refreshed every hour

7.3 MLflow Artifact Fallback Chain

A production-grade artifact retrieval strategy was implemented to ensure the application always loads the best available models, even if recent training runs failed or produced incomplete artifacts:

- Query all runs from the Default MLflow experiment, ordered by start time descending (newest first).
- For each run, check two conditions: (1) run status must equal `FINISHED`, and (2) all six required artifact files must be present in the run's artifact store.
- The first run that passes both checks is selected. Runs that are `FAILED`, `RUNNING`, or missing artifacts are logged as warnings and skipped automatically.
- If no valid run is found after exhausting all runs, a `RuntimeError` is raised with a descriptive message.

This pattern ensures that a broken training run (e.g., interrupted by a timeout or API failure) never causes the application to load corrupt or incomplete model files.

8. Challenges Encountered and Resolutions

The following technical blockers were encountered during development. Each is documented with its root cause and the resolution applied, in accordance with the project's engineering log.

Blocker 1: Historical Weather Data Access Limitations

Symptom: During the initial backfilling phase, attempts to retrieve historical hourly weather data for the training dataset were rejected with HTTP 401 Unauthorized errors.

Root Cause: The pipeline was initially architected using the OpenWeather API for all meteorological data. While the current-weather endpoint is available on their free tier, access to their historical weather API requires a paid enterprise subscription.

Resolution: The historical data ingestion pipeline was re-architected to use the Open-Meteo API, which provides free, open-source access to extensive historical weather archives. This ensured the project could generate a rich, multi-year training dataset without incurring API costs.

Blocker 2: Hopsworks Feature Store Materialisation Failures

Symptom: The training pipeline consistently failed during the data retrieval stage, hanging indefinitely when attempting to create training data splits or raising feature group materialisation timeouts.

Root Cause: The project initially utilised Hopsworks as the central Feature Store. However, the Hopsworks free-tier cluster experienced severe latency and materialisation instability when attempting to join and serve the hourly feature groups, blocking the automated GitHub Actions pipeline.

Resolution: The MLOps architecture was completely decoupled and migrated to more stable cloud providers. MongoDB Atlas was implemented as the Feature Store (storing pre-joined feature vectors as documents to bypass complex SQL joins), and DagsHub MLflow was adopted as the Model Registry. This migration permanently resolved the pipeline instability.

Blocker 3: Random Forest Underperformance on Tabular Time-Series

Symptom: During early model experimentation, a Random Forest Regressor was utilised as the primary ensemble model, but it struggled to capture sudden spikes in the AQI and produced elevated MAE scores on the Day 2 and Day 3 horizons.

Root Cause: Random Forest models rely on bagging (averaging independent decision trees), which limits their ability to accurately extrapolate extreme, non-linear spikes in time-series data (such as winter temperature inversions in Karachi).

Resolution: The Random Forest model was replaced with XGBoost (Gradient Boosted Trees). Because XGBoost trains trees sequentially to explicitly correct the residual errors of previous trees, it proved far superior at predicting extreme AQI fluctuations. XGBoost ultimately achieved the best performance metrics and was promoted to the primary inference model.

Blocker 4: KeyError on Cyclical Feature Columns at Inference

Symptom: Cell 16 of the EDA notebook raised `KeyError: 'hour_cos'` when attempting to access cyclical columns from the MongoDB-fetched `DataFrame`.

Root Cause: Cyclical features (`hour_sin`, `hour_cos`, `month_sin`, `month_cos`, `dow_sin`, `dow_cos`) are computed inside the `ColumnTransformer` during training but are never written back to MongoDB. The feature store only contains the raw temporal integers (`hour`, `month`, `day_of_week`).

Resolution: Cyclical encodings are now reconstructed on the fly at both EDA time and inference time using the raw temporal columns that are stored in MongoDB. The formula $\sin(2\pi \times \text{value} / \text{period})$ is applied directly in Python before constructing the feature vector.

Critical Implication: This pattern must be applied consistently in every inference path. Any code that attempts to read cyclical features directly from MongoDB will fail.

Blocker 5: Incorrect Model Champion Assumption

Symptom: Initial Section 6 plan assumed XGBoost would be the clear winner across all metrics and horizons.

Root Cause: The `metrics.json` output revealed that Ridge Regression wins Day 1 R^2 (0.316 vs 0.304) and ties Day 1 MAE within a negligible 0.36 AQI point margin. No single model dominates all three horizons on all metrics.

Resolution: Both XGBoost and Ridge were framed as co-leaders in the analysis. XGBoost was selected as the featured application model because it wins 2 of 3 horizons on MAE (the most user-interpretable metric) and achieves the highest R^2 at Day 3. This decision is explicitly documented in the dashboard's Why XGBoost callout section.

Blocker 6: MLflow Tracking URI Returns None on Hugging Face

Symptom: Container logs showed MLflow configured. Tracking URI: `None`, followed by `RuntimeError: No MLflow runs found in experiment 'Default'`.

Root Cause: The `MLFLOW_TRACKING_URI` constant was constructed at module import time using `os.getenv('DAGSHUB_USERNAME')`. On Hugging Face Spaces, environment variables injected as Secrets are not available at import time — only at function call time. The URI was therefore set to a string containing `None` before the secrets were loaded.

Resolution: The URI construction was moved inside the `_configure_mlflow()` function, which is called at runtime rather than at import time. This ensures the environment variables are resolved only when the function executes, by which time all HF Secrets have been injected.

9. Results Summary

9.1 Model Performance Overview

Karachi AQI — Model Performance & SHAP Analysis

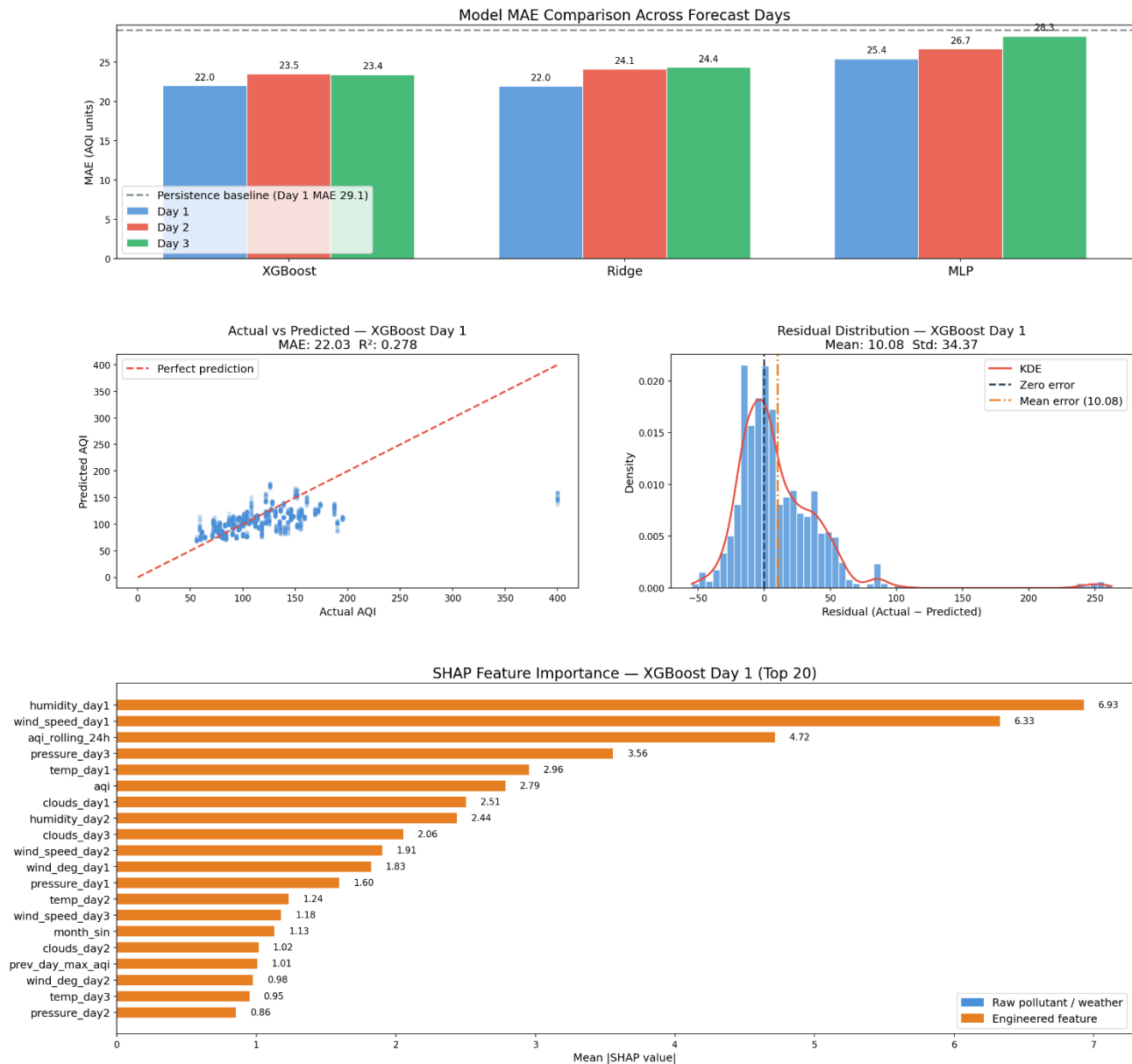


Figure 7: SHAP summary plot showing global feature importance for the XGBoost model.

All three ML models decisively outperformed the persistence baseline across all forecast horizons and all three metrics. The key quantitative results are:

- XGBoost achieved the best overall performance on the metric most relevant to users (MAE), winning Day 2 (23.78) and Day 3 (23.22) and achieving the highest R² at Day 3 (0.322).
- Ridge Regression demonstrated competitive performance, particularly at Day 1 where it achieved the lowest MAE (21.98) and highest R² (0.316), making it a strong candidate for near-term forecasting.

- MLP, while outperforming the persistence baseline, showed degrading performance at longer horizons (Day 3 $R^2 = 0.120$), suggesting that the available feature set does not provide sufficient signal for deep learning to exploit additional non-linear structure beyond what tree-based models capture.
- Persistence baseline achieved negative R^2 values at all horizons (-0.016, -0.014, -0.013), confirming that even the simplest naive forecast based on the current AQI value has no predictive power beyond the dataset mean — a strong validation that ML-based forecasting provides genuine value.

9.2 Live System Verification

The deployed system was verified against real-world IQAir US AQI readings for Karachi on 7 June 2026. IQAir reported a current AQI of approximately 79 (Moderate). The model predictions (XGBoost Day 1: 113.2, Ridge Day 1: 103.6) showed an over-prediction of approximately 30-35 AQI points, attributable to data freshness lag: the latest MongoDB timestamp at time of comparison was 2026-06-05 23:00 UTC, meaning the model was operating on features that were approximately 38 hours stale.

10. Conclusions and Future Work

10.1 Conclusions

This project successfully delivered a complete, production-grade air quality intelligence system for Karachi within the internship timeline. All seven project objectives were met:

- A fully automated feature pipeline ingests hourly data from two external APIs and stores engineered features in MongoDB Atlas.
- A training pipeline trains three machine learning models daily, logs all results to DagsHub MLflow, and uploads validated artifacts automatically.
- GitHub Actions provides zero-touch CI/CD automation for both pipelines, running continuously without manual intervention.
- XGBoost was selected as the primary model through rigorous multi-metric, multi-horizon evaluation, with Ridge Regression as a competitive co-leader.
- All ML models demonstrated genuine predictive value, comprehensively outperforming the persistence baseline with positive R^2 versus the baseline's negative R^2 .
- The interactive Streamlit dashboard is publicly deployed on Hugging Face Spaces, providing real-time AQI monitoring and 3-day forecasts accessible via a stable public URL.
- A production-grade MLflow fallback chain ensures the application always serves the best validated model artifacts, even in the presence of failed training runs.

10.2 Future Work

Several improvements would further enhance the system's accuracy and utility:

- Incorporate additional data sources such as satellite-derived aerosol optical depth (AOD) data, which has demonstrated strong correlations with ground-level PM_{2.5} in South Asian urban environments.
- Implement a time-series aware cross-validation strategy (walk-forward validation) to more accurately represent the model's real-world performance on unseen future data.
- Explore LSTM or Transformer architectures that can leverage the sequential nature of the time series more effectively than the current MLP formulation.
- Add SHAP-based feature importance to the live dashboard to increase interpretability for non-technical users, showing which environmental factors are driving each forecast.
- Implement AQI threshold alerts that notify users via email or SMS when the forecast exceeds Unhealthy thresholds — a feature requested in the original project brief.
- Migrate from hourly GitHub Actions polling to an event-driven ingestion architecture to reduce latency between data availability and dashboard update.

Appendix A: Technology Stack Summary

Category	Technology	Version	Purpose
Data Ingestion	Open-Meteo API	v1	3-day weather forecast at inference
Feature Store	MongoDB Atlas	7.x	Cloud document database
ML Framework	XGBoost	3.2.0	Gradient boosted trees
ML Framework	Scikit-learn	1.8.0	Ridge regression + StandardScaler
ML Framework	Keras/TensorFlow CPU	3.14/2.21	MLP neural network
Tracking	MLflow	3.13.0	Experiment tracking + artifact store
Tracking	DagsHub	0.7.0	MLflow hosting + Git integration
Dashboard	Streamlit	1.58.0	Interactive web application
Visualisation	Plotly	6.8.0	Interactive charts
CI/CD	GitHub Actions	N/A	Pipeline automation
Deployment	Hugging Face Spaces	Docker	Public cloud hosting
Data Processing	Pandas	2.3.3	DataFrame operations
Data Processing	NumPy	2.4.4	Numerical computations
Model Persistence	Joblib	1.5.3	Pickle-based model serialisation

Appendix B: Complete Feature List (43 Features)

#	Feature	Category	Source
1	aqi	Direct	MongoDB
2	aqi_change_rate	Direct	MongoDB
3	clouds	Direct	MongoDB
4	co	Direct	MongoDB
5	humidity	Direct	MongoDB
6	is_weekend	Direct	MongoDB
7	no2	Direct	MongoDB
8	o3	Direct	MongoDB

#	Feature	Category	Source
9	pm10	Direct	MongoDB
10	pm2_5	Direct	MongoDB
11	pm2_5_rolling_3h	Direct	MongoDB
12	pressure	Direct	MongoDB
13	so2	Direct	MongoDB
14	temp	Direct	MongoDB
15	wind_deg	Direct	MongoDB
16	wind_speed	Direct	MongoDB
17	hour_sin	Cyclical	Computed on-the-fly
18	hour_cos	Cyclical	Computed on-the-fly
19	month_sin	Cyclical	Computed on-the-fly
20	month_cos	Cyclical	Computed on-the-fly
21	dow_sin	Cyclical	Computed on-the-fly
22	dow_cos	Cyclical	Computed on-the-fly
23	prev_day_max_aqi	Windowed	48h context window
24	prev_2day_max_aqi	Windowed	48h context window
25	aqi_rolling_24h	Windowed	48h context window
26	temp_day1	Forecast	Open-Meteo
27	temp_day2	Forecast	Open-Meteo
28	temp_day3	Forecast	Open-Meteo
29	humidity_day1	Forecast	Open-Meteo
30	humidity_day2	Forecast	Open-Meteo
31	humidity_day3	Forecast	Open-Meteo
32	pressure_day1	Forecast	Open-Meteo
33	pressure_day2	Forecast	Open-Meteo
34	pressure_day3	Forecast	Open-Meteo
35	wind_speed_day1	Forecast	Open-Meteo
36	wind_speed_day2	Forecast	Open-Meteo
37	wind_speed_day3	Forecast	Open-Meteo
38	wind_deg_day1	Forecast	Open-Meteo
39	wind_deg_day2	Forecast	Open-Meteo
40	wind_deg_day3	Forecast	Open-Meteo

#	Feature	Category	Source
41	clouds_day1	Forecast	Open-Meteo
42	clouds_day2	Forecast	Open-Meteo
43	clouds_day3	Forecast	Open-Meteo